

The Complete Textbook

From Fundamentals to Production

Comprehensive Coverage of:

- ◆ Architecture & Core Concepts ◆
 - ◆ Workloads & Scheduling ◆
 - ◆ Networking & Services ◆
 - ◆ Storage & Configuration ◆
 - ◆ Security & RBAC ◆
 - ◆ Observability & Monitoring ◆
- ◆ Helm & Package Management ◆
 - ◆ CI/CD & GitOps ◆
- ◆ Production Best Practices ◆

tech__talks

Table of Contents

Table of Contents	2
Chapter 1: Introduction to Kubernetes	5
1.1 What is Kubernetes?	5
1.2 Brief History & Evolution	5
1.3 Why Kubernetes? Problems It Solves	5
1.4 Key Concepts & Terminology	6
1.5 Kubernetes vs Docker Swarm vs Nomad	6
Chapter 2: Kubernetes Architecture	7
2.1 The Control Plane	7
kube-apiserver	7
etcd	7
kube-scheduler	7
kube-controller-manager	8
2.2 Worker Nodes	8
kubelet	8
kube-proxy	8
Container Runtime	9
Chapter 3: Setting Up Kubernetes	10
3.1 Local Development Options	10
3.2 Installing minikube	10
3.3 Installing kubectl	10
3.4 Managed Kubernetes Services	10
3.5 kubectl — Self-Managed Clusters	11
Chapter 4: Pods — The Atomic Unit	12
4.1 Pod Anatomy	12
4.2 Creating Your First Pod	12
4.3 Pod Lifecycle	13
4.4 Init Containers	13
4.5 Sidecar Containers	13
4.6 Resource Requests and Limits	13
Chapter 5: Workload Resources	15
5.1 Deployments	15
5.2 ReplicaSets	16
5.3 StatefulSets	16
5.4 DaemonSets	16
5.5 Jobs and CronJobs	16

5.6 Horizontal Pod Autoscaler (HPA)	17
Chapter 6: Networking in Kubernetes	18
6.1 Kubernetes Networking Model.....	18
6.2 Services	18
ClusterIP (Default)	18
NodePort	18
LoadBalancer	18
ExternalName	18
6.3 DNS in Kubernetes.....	19
6.4 Ingress	19
6.5 Network Policies.....	20
Chapter 7: Storage in Kubernetes.....	21
7.1 Volumes	21
7.2 PersistentVolumes (PV) and PersistentVolumeClaims (PVC).....	21
7.3 StorageClasses and Dynamic Provisioning	22
7.4 Access Modes	22
Chapter 8: Configuration Management	23
8.1 ConfigMaps	23
8.2 Secrets	23
8.3 Environment Variables	24
Chapter 9: Security in Kubernetes	25
9.1 Authentication	25
9.2 RBAC (Role-Based Access Control).....	25
9.3 Pod Security	26
9.4 Security Contexts	26
Chapter 10: Scheduling & Node Management	27
10.1 Labels and Selectors	27
10.2 Taints and Tolerations	27
10.3 Node Affinity.....	27
10.4 Pod Affinity and Anti-Affinity	28
10.5 Resource Quotas and LimitRanges	28
Chapter 11: Observability — Logging, Metrics & Tracing	29
11.1 Logging Architecture	29
11.2 Metrics with Prometheus	29
11.3 Grafana Dashboards	29
11.4 Distributed Tracing with Jaeger/Zipkin	29
11.5 Health Probes	30
Chapter 12: Helm — Kubernetes Package Manager	31

12.1 Helm Concepts.....	31
12.2 Essential Helm Commands	31
12.3 Creating a Custom Chart.....	31
Chapter 13: CI/CD and GitOps with Kubernetes	33
13.1 CI/CD Pipeline Overview	33
13.2 GitHub Actions for Kubernetes	33
13.3 GitOps with ArgoCD	33
13.4 FluxCD — Alternative GitOps	34
Chapter 14: Advanced Topics.....	35
14.1 Custom Resource Definitions (CRDs)	35
14.2 Kubernetes Operators	35
14.3 Service Mesh	35
14.4 Multi-Cluster Management	36
14.5 Vertical Pod Autoscaler (VPA).....	36
14.6 Cluster Autoscaler	36
Chapter 15: Production Best Practices	37
15.1 Reliability Best Practices	37
15.2 Security Best Practices.....	37
15.3 Cost Optimization	37
15.4 Upgrade Strategy	37
15.5 Disaster Recovery	38
15.6 Kubectl Productivity Tips	38

Chapter 1: Introduction to Kubernetes

Kubernetes (often abbreviated as K8s) is an open-source container orchestration platform originally developed by Google and donated to the Cloud Native Computing Foundation (CNCF) in 2014. It automates the deployment, scaling, and management of containerized applications across clusters of machines.

1.1 What is Kubernetes?

Kubernetes is a production-grade container orchestration system. Rather than running containers directly on individual servers, Kubernetes acts as the intelligent brain that decides where, when, and how containers should run across a fleet of machines.

The name Kubernetes comes from the Greek word for 'helmsman' or 'pilot' — the one who steers the ship. The logo represents a ship's helm, reinforcing this nautical theme that runs throughout the ecosystem (Docker uses a whale, Helm uses a ship's helm, etc.).

□ DIAGRAM — Kubernetes High-Level Architecture

Draw a large rectangle labeled 'Kubernetes Cluster'. Inside, place two regions: LEFT box = 'Control Plane' (contains etcd, API Server, Scheduler, Controller Manager). RIGHT boxes = 3 'Worker Node' boxes each containing a Kubelet, Kube-proxy, and 2-3 Pod icons. Draw arrows from Control Plane → Worker Nodes labeled 'orchestration'. Add 'kubectl' icon outside cluster pointing to API Server.

1.2 Brief History & Evolution

Kubernetes grew out of Google's internal cluster manager called Borg, which had been managing billions of containers a week for over a decade. Google open-sourced Kubernetes in June 2014 as a way to share the lessons learned from building Borg.

- 2014 — Google open-sources Kubernetes v0.1
- 2015 — Kubernetes v1.0 released; CNCF formed; Kubernetes donated to CNCF
- 2016 — Helm package manager introduced; Kubernetes 1.5 with stable API
- 2017 — Docker adds native Kubernetes support; major cloud vendors launch managed K8s
- 2018 — Kubernetes 1.10+; CNCF graduation; becomes de-facto container orchestrator
- 2020+ — Kubernetes dominates enterprise infrastructure; millions of clusters worldwide

1.3 Why Kubernetes? Problems It Solves

Before container orchestration, deploying and managing microservices at scale was painful. Kubernetes directly addresses these challenges:

Problem	Kubernetes Solution
Manual container placement	Automated scheduling based on resource requirements
No self-healing	Auto-restarts failed containers; reschedules from failed nodes
Manual scaling	Horizontal Pod Autoscaler scales based on CPU/memory/custom metrics

Service discovery complexity	Built-in DNS and Service objects for stable endpoints
Rolling updates are risky	Declarative rollout with zero-downtime and easy rollback
Configuration sprawl	ConfigMaps and Secrets for environment-specific configuration
Storage management	Persistent Volumes abstract underlying storage providers

1.4 Key Concepts & Terminology

Before diving deep, let's establish the core vocabulary:

- **Cluster** — A set of machines (nodes) running Kubernetes
- **Node** — A single machine (VM or physical) in the cluster
- **Pod** — The smallest deployable unit; one or more containers
- **Deployment** — Manages a set of identical Pods; handles updates
- **Service** — Stable network endpoint to access a group of Pods
- **Namespace** — Virtual cluster within a cluster for isolation
- **Control Plane** — The brains of the cluster managing state
- **kubectl** — Command-line tool to interact with Kubernetes

1.5 Kubernetes vs Docker Swarm vs Nomad

Feature	Kubernetes	Docker Swarm	HashiCorp Nomad
Learning Curve	Steep	Low	Medium
Ecosystem	Massive (CNCF)	Limited	Growing
Auto-scaling	Built-in HPA/VPA	Manual	Built-in
Multi-workload	Containers only	Containers only	Containers + VMs + binaries
Managed Cloud Support	All major clouds	Limited	Limited
Production Adoption	Very High	Declining	Niche

Chapter 2: Kubernetes Architecture

Understanding Kubernetes architecture is fundamental to operating it effectively. Kubernetes follows a master-worker architecture where the Control Plane manages the cluster state and Worker Nodes run the actual application workloads.

2.1 The Control Plane

The Control Plane (formerly called 'Master') is the nervous system of Kubernetes. It makes global decisions about the cluster (scheduling) and detects/responds to cluster events. In production, the Control Plane runs on dedicated nodes with high availability (3 or 5 replicas).

□ DIAGRAM — Control Plane Deep Dive

Draw a rectangle labeled 'Control Plane'. Inside, place 4 component boxes: (1) 'kube-apiserver' — center, largest box, all arrows pass through it; (2) 'etcd' — right side, cylindrical database shape, labeled 'Distributed Key-Value Store'; (3) 'kube-scheduler' — top left box; (4) 'kube-controller-manager' — bottom left box with 4 smaller icons inside (Node Controller, Deployment Controller, Endpoint Controller, ServiceAccount Controller). Draw bidirectional arrows from all components to kube-apiserver. Label etcd arrow as 'Read/Write cluster state'.

kube-apiserver

The API Server is the front-end of the Kubernetes Control Plane. It is the only component that directly interacts with etcd. Every operation (kubectl, controllers, kubelet communication) goes through the API Server.

- Exposes the Kubernetes REST API
- Validates and processes API requests
- Is stateless — can be scaled horizontally
- Authenticates, authorizes, and admission-controls every request

etcd

etcd is a distributed, consistent, and highly-available key-value store used as Kubernetes' backing store for all cluster data. Everything in Kubernetes — Pods, Services, ConfigMaps, Secrets — is stored in etcd as JSON/Protobuf.

- Uses the Raft consensus algorithm for distributed consistency
- The ONLY stateful component in the control plane
- CRITICAL: Always backup etcd! Loss of etcd = loss of cluster state
- Typical deployment: 3 or 5 etcd nodes for HA (always odd number for quorum)

□ **NOTE:** etcd stores cluster state, NOT application data. Your application's database is separate and lives in PersistentVolumes.

kube-scheduler

The Scheduler watches for newly created Pods that have no assigned node and selects an appropriate node for them to run on. Scheduling decisions consider multiple factors:

- Resource requirements (CPU, memory requests)
- Hardware/software constraints (node selectors, taints/tolerations)
- Affinity and anti-affinity rules
- Data locality for stateful workloads
- Inter-workload interference

kube-controller-manager

The Controller Manager runs all the built-in Kubernetes controllers in a single process. Each controller is a control loop that watches the current state and works to bring it to the desired state.

Controller	Responsibility
Node Controller	Notifies and responds when nodes go down
Replication Controller	Maintains correct number of Pod replicas
Endpoints Controller	Populates Endpoint objects (joins Services & Pods)
Service Account Controller	Creates default service accounts for new namespaces
Deployment Controller	Manages Deployment rollouts and rollbacks
Job Controller	Watches for completed Jobs

2.2 Worker Nodes

Worker Nodes (previously called 'Minions') are the machines where your application Pods actually run. Each worker node contains three essential components:

□ DIAGRAM — Worker Node Components

Draw a rectangle labeled 'Worker Node'. Inside place 3 main components: (1) 'kubelet' — top section, with arrows to Pod icons; (2) 'kube-proxy' — middle section, with iptables/ipvs icon and arrow to network; (3) 'Container Runtime' — bottom section, showing Docker/containerd/CRI-O icons. Above the node, show the Control Plane API Server sending commands down to the kubelet via arrows. Show multiple Pod boxes (each containing container icons) running on the node.

kubelet

The kubelet is an agent that runs on each node and ensures containers described in PodSpecs are running and healthy. It communicates with the API Server and manages the node's containers.

- Registers the node with the Kubernetes API Server
- Watches for PodSpecs assigned to its node
- Ensures containers match their specifications
- Reports node and Pod status back to the API Server
- Performs liveness and readiness probe checks

kube-proxy

kube-proxy runs on every node and implements the Kubernetes Service concept by maintaining network rules. It enables Pods to communicate with each other and with the outside world.

- Maintains network rules on nodes using iptables or IPVS
- Handles traffic forwarding for ClusterIP services
- Implements load balancing across Pod endpoints

Container Runtime

The container runtime is the software responsible for running containers. Kubernetes supports any runtime that implements the Container Runtime Interface (CRI):

- containerd — Most widely used; originally part of Docker
- CRI-O — Lightweight runtime designed for Kubernetes
- Docker Engine — Deprecated in K8s 1.24+ (dockershim removed)

Note: Docker images still work perfectly — it's just the runtime layer that changed.

Chapter 3: Setting Up Kubernetes

There are multiple ways to install and run Kubernetes depending on your use case: from local development environments to managed cloud services to on-premise production clusters.

3.1 Local Development Options

Tool	Best For
minikube	Beginners; runs full K8s in a VM or container
kind (Kubernetes IN Docker)	CI pipelines; fast cluster creation using Docker containers
k3d	Lightweight K3s inside Docker; fast for development
Docker Desktop	Mac/Windows developers; built-in K8s toggle
Rancher Desktop	Open-source Docker Desktop alternative with K8s

3.2 Installing minikube

minikube is the recommended way to learn Kubernetes locally. It creates a single-node cluster on your machine:

```
# macOS
brew install minikube

# Linux
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Start cluster
minikube start --driver=docker --cpus=2 --memory=4g
minikube status
```

3.3 Installing kubectl

kubectl is the primary CLI tool for interacting with Kubernetes clusters:

```
# macOS
brew install kubectl

# Linux
curl -LO "https://dl.k8s.io/release/$(curl -sL
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# Verify
kubectl version --client
```

3.4 Managed Kubernetes Services

For production workloads, managed Kubernetes services handle the Control Plane for you, letting you focus on applications:

Cloud Provider	Service Name
Amazon Web Services	EKS (Elastic Kubernetes Service)
Google Cloud Platform	GKE (Google Kubernetes Engine)
Microsoft Azure	AKS (Azure Kubernetes Service)
DigitalOcean	DOKS (DigitalOcean Kubernetes Service)
Oracle Cloud	OKE (Oracle Container Engine for Kubernetes)
IBM Cloud	IKS (IBM Kubernetes Service)

3.5 kubeadm — Self-Managed Clusters

kubeadm is the official tool for bootstrapping production-grade Kubernetes clusters on your own infrastructure:

```
# On control plane node
sudo kubeadm init --pod-network-cidr=10.244.0.0/16

# Configure kubectl
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config

# Install CNI (Flannel)
kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml

# Join worker nodes (use token from kubeadm init output)
sudo kubeadm join <control-plane-ip>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Chapter 4: Pods — The Atomic Unit

Pods are the smallest and most basic deployable objects in Kubernetes. A Pod represents a single instance of a running process and encapsulates one or more containers, storage resources, a unique network IP, and options governing how the container(s) should run.

4.1 Pod Anatomy

Every Pod has a unique IP address within the cluster. All containers within a Pod share the same network namespace (IP and port space) and can communicate via localhost. They also share storage volumes.

□ DIAGRAM — Pod Internal Structure

Draw a large rectangle labeled 'Pod'. Inside show: (1) A 'Shared Network Namespace' section showing an IP address (e.g., 10.244.1.5) with a network icon; (2) Two Container rectangles side-by-side labeled 'Main Container' and 'Sidecar Container', each with a small Docker icon; (3) A 'Shared Volume' cylinder at the bottom connecting to both containers with arrows labeled 'mount'. Show that containers can communicate via 'localhost' with a double-headed arrow between them.

4.2 Creating Your First Pod

Pods can be created imperatively (via kubectl commands) or declaratively (via YAML manifests). Declarative is always preferred in production:

```
# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-first-pod
  labels:
    app: myapp
    tier: frontend
spec:
  containers:
  - name: nginx-container
    image: nginx:1.25
    ports:
    - containerPort: 80
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "500m"
        memory: "256Mi"
```

```
# Apply the manifest
kubectl apply -f pod.yaml
kubectl get pods
kubectl describe pod my-first-pod
```

```
kubectl logs my-first-pod
```

4.3 Pod Lifecycle

Pods go through several lifecycle phases:

Phase	Description
Pending	Pod accepted by cluster but containers not yet running (scheduling/image pull)
Running	Pod bound to a node; at least one container is running
Succeeded	All containers terminated successfully with exit code 0
Failed	All containers terminated; at least one container failed
Unknown	Pod state cannot be determined (node communication error)
CrashLoopBackOff	Container keeps crashing; Kubernetes backs off restart timing
ImagePullBackOff	Cannot pull the container image (wrong name, no auth, rate limit)

4.4 Init Containers

Init containers run before app containers start. They run to completion sequentially and must all succeed before the main containers start. Common use cases: run database migrations, wait for a service to be ready, clone a Git repo.

```
spec:
  initContainers:
    - name: wait-for-db
      image: busybox
      command: ['sh', '-c', 'until nc -z db-service 5432; do sleep 2; done']
  containers:
    - name: app
      image: myapp:latest
```

4.5 Sidecar Containers

Sidecar containers run alongside the main container in the same Pod. They extend or enhance the main container without modifying it. Common patterns:

- Log shipper — Reads logs and sends to centralized logging (Fluentd, Filebeat)
- Proxy — Service mesh sidecar like Envoy (Istio, Linkerd)
- Adapter — Transforms data format between the app and external services
- Ambassador — Proxies network access to the outside world

4.6 Resource Requests and Limits

Properly setting resource requests and limits is critical for cluster stability and workload performance:

- Requests — What the scheduler uses for placement decisions. The node must have at least this much available.
- Limits — The maximum amount a container can use. CPU is throttled; memory limit breach causes OOMKill.

□ **NOTE:** Always set requests and limits. Without requests, the scheduler may overcommit nodes. Without limits, one container can starve others.

Chapter 5: Workload Resources

While Pods are the basic unit, you rarely create them directly. Instead, you use higher-level workload resources that manage Pods for you, providing self-healing, scaling, and update capabilities.

5.1 Deployments

Deployments are the most common workload resource. They describe a desired state for Pods and ReplicaSets, and the Deployment Controller works to match the actual state to the desired state.

□ DIAGRAM — Deployment → ReplicaSet → Pod Hierarchy

Draw three levels: TOP: 'Deployment' box (blue) with label 'Desired: 3 replicas, Image: nginx:1.25'. MIDDLE: 'ReplicaSet' box (teal) connected by arrow from Deployment, labeled 'Ensures 3 Pods exist'. BOTTOM: Three 'Pod' boxes connected from ReplicaSet by arrows. Each Pod shows 'nginx:1.25' container inside. On the right side, show an old ReplicaSet (grey, 0 replicas) from a previous deployment.

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

```
# Key Deployment commands
kubectl rollout status deployment/nginx-deployment
kubectl rollout history deployment/nginx-deployment
kubectl rollout undo deployment/nginx-deployment
kubectl set image deployment/nginx-deployment nginx=nginx:1.26
```

5.2 ReplicaSets

A ReplicaSet ensures that a specified number of Pod replicas are running at any time. While you can use ReplicaSets directly, it's recommended to use Deployments instead, which manage ReplicaSets and provide update capabilities on top.

5.3 StatefulSets

StatefulSets are designed for stateful applications that require stable network identities and persistent storage. Unlike Deployments, StatefulSets maintain a sticky identity for each Pod.

- Pods get stable, predictable names: pod-0, pod-1, pod-2
- Pods are created in order (0, 1, 2) and deleted in reverse (2, 1, 0)
- Each Pod gets its own PersistentVolumeClaim
- Use cases: databases (MySQL, PostgreSQL, Cassandra), Kafka, Zookeeper, Elasticsearch

□ DIAGRAM — StatefulSet vs Deployment

Draw two columns side-by-side. LEFT column labeled 'Deployment (Stateless)': Show 3 Pods all identical, labeled 'pod-abc12', 'pod-def34', 'pod-ghi56' (random names). Show one shared PVC connecting to all pods. RIGHT column labeled 'StatefulSet (Stateful)': Show 3 Pods labeled 'mysql-0', 'mysql-1', 'mysql-2' (ordered names). Show each pod has its OWN PVC (three separate PVCs). Add a label 'Stable Network Identity' under StatefulSet column.

5.4 DaemonSets

A DaemonSet ensures that all (or some) Nodes run a copy of a Pod. As nodes are added to the cluster, Pods are added to them automatically. Typical use cases:

- Log collection — Fluentd, Filebeat on every node
- Monitoring agents — Prometheus node-exporter, Datadog agent
- Storage daemons — Ceph, GlusterFS cluster daemons
- Network plugins — CNI plugins, kube-proxy itself

5.5 Jobs and CronJobs

Jobs manage batch workloads that run to completion. CronJobs schedule Jobs on a time-based schedule.

```
# CronJob example — runs every hour
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hourly-report
spec:
  schedule: "0 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
```



```
- name: reporter
  image: report-generator:latest
  restartPolicy: OnFailure
```

5.6 Horizontal Pod Autoscaler (HPA)

HPA automatically scales the number of Pod replicas based on observed CPU utilization, memory utilization, or custom metrics:

```
kubectl autoscale deployment nginx-deployment --cpu-percent=50 --min=2 --max=10
```

```
# Or declaratively:
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Chapter 6: Networking in Kubernetes

Kubernetes networking follows four fundamental rules: every Pod gets its own IP, all Pods can communicate without NAT, all nodes can communicate with all Pods, and the IP a Pod sees itself is the same as others see it.

6.1 Kubernetes Networking Model

□ DIAGRAM — Kubernetes Networking Overview

Draw a cluster outline. Show 3 Worker Nodes, each containing 2-3 Pods with unique IPs in the 10.244.x.x range. Draw a network overlay connecting all nodes labeled 'Pod Network / CNI (e.g., Flannel)'. Show a 'Service' box (ClusterIP: 10.96.x.x) with arrows pointing to multiple Pods. Outside the cluster, show an 'Ingress Controller' box receiving external traffic (port 80/443) and routing to Services. Label the layers: 'Node Network', 'Pod Network', 'Service Network'.

6.2 Services

Services provide a stable network endpoint for a set of Pods. Since Pods are ephemeral (they come and go), Services provide stable DNS names and IP addresses.

ClusterIP (Default)

Exposes the Service on a cluster-internal IP. Only reachable within the cluster.

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 8080
```

NodePort

Exposes the Service on each Node's IP at a static port (30000-32767). Accessible from outside the cluster using NodeIP:NodePort.

LoadBalancer

Exposes the Service externally using a cloud provider's load balancer. The external load balancer routes traffic to the NodePort Service. Each LoadBalancer Service gets a unique external IP.

ExternalName

Maps a Service to an external DNS name. Returns a CNAME record rather than proxying traffic. Useful for integrating external services into the cluster's DNS.

Service Type	Accessible From	Use Case
ClusterIP	Within cluster only	Internal microservices communication
NodePort	Node IP + Port	Development/testing external access
LoadBalancer	External LB IP	Production external access on cloud
ExternalName	Within cluster (DNS)	Map internal name to external service
Headless (clusterIP: None)	Direct Pod IPs	StatefulSets, direct Pod addressing

6.3 DNS in Kubernetes

CoreDNS is the default DNS server in Kubernetes. It automatically creates DNS records for Services and (optionally) Pods. DNS naming follows this pattern:

- <service-name>.<namespace>.svc.cluster.local — Full FQDN for a Service
- <service-name> — Works within the same namespace
- <service-name>.<namespace> — Cross-namespace access

```
# Example DNS resolution
# Service 'my-db' in namespace 'production'
my-db                      # from same namespace
my-db.production          # from another namespace
my-db.production.svc.cluster.local # full FQDN
```

6.4 Ingress

Ingress manages external access to Services within a cluster, typically HTTP/HTTPS. It provides load balancing, SSL termination, and name-based virtual hosting. Ingress requires an Ingress Controller (nginx, traefik, HAProxy, etc.) to be deployed.

□ DIAGRAM — Ingress Traffic Flow

Draw external internet traffic (browser icon) going to 'Ingress Controller' box. From Ingress Controller, draw two routing paths: PATH 1: traffic with path /api/* → 'api-service' → 3 API Pods. PATH 2: traffic with path /* → 'frontend-service' → 3 Frontend Pods. Above the routing paths, show 'SSL Termination' happening at the Ingress Controller. Label the host as 'myapp.example.com'.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  tls:
```

```
- hosts: [myapp.example.com]
  secretName: myapp-tls
rules:
- host: myapp.example.com
  http:
    paths:
    - path: /api
      pathType: Prefix
      backend:
        service:
          name: api-service
          port:
            number: 8080
```

6.5 Network Policies

NetworkPolicies act as a firewall for Pod-to-Pod communication. By default, all Pods in a cluster can communicate with all other Pods. NetworkPolicies restrict this.

```
# Deny all ingress to pods with label app=backend
# Allow only from pods with label app=frontend
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-only
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
```

Chapter 7: Storage in Kubernetes

Container file systems are ephemeral — when a container restarts, all data is lost. Kubernetes provides a rich storage subsystem to handle persistent data for stateful applications.

7.1 Volumes

A Volume is a directory accessible to containers in a Pod. Unlike Docker volumes, Kubernetes volumes have a lifetime tied to the Pod. Common volume types:

Volume Type	Description
emptyDir	Temporary scratch space; exists as long as Pod exists
hostPath	Mounts a file/directory from the host node
configMap	Mount ConfigMap data as files
secret	Mount Secret data as files
persistentVolumeClaim	Mount a PersistentVolume into a Pod
nfs	Mount an NFS share
awsElasticBlockStore	Mount AWS EBS volume
azureDisk	Mount Azure Managed Disk

7.2 PersistentVolumes (PV) and PersistentVolumeClaims (PVC)

PersistentVolumes decouple storage provisioning from storage consumption. The PV is the actual storage resource; the PVC is a request for storage by a user.

□ DIAGRAM — PV / PVC Binding Flow

Draw three stages: STAGE 1 (Admin): 'Admin' icon → creates 'PersistentVolume' (100Gi, ReadWriteOnce, NFS). STAGE 2 (Binding): Draw a 'PersistentVolumeClaim' box (50Gi request) with a bidirectional arrow labeled 'Bound' connecting to the PV. STAGE 3 (Usage): 'Pod' box mounts the PVC as a volume via an arrow labeled 'volumeMount'. Add a small 'StorageClass' box at the top labeled 'Dynamic Provisioning (optional)' with a dotted arrow to PV showing automatic creation.

```
# PersistentVolume (created by admin)
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-100g
spec:
  capacity:
    storage: 100Gi
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  nfs:
    server: nfs-server.example.com
    path: /exports/data
```

```
# PersistentVolumeClaim (created by developer)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-app-pvc
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 20Gi
```

7.3 StorageClasses and Dynamic Provisioning

StorageClasses allow dynamic provisioning of PersistentVolumes. Instead of an admin pre-creating PVs, when a PVC is created Kubernetes automatically provisions a PV using the StorageClass provisioner.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
  iops: "3000"
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

7.4 Access Modes

Access Mode	Abbreviation	Description
ReadWriteOnce	RWO	Single node read/write (most common)
ReadOnlyMany	ROX	Many nodes read-only
ReadWriteMany	RWX	Many nodes read/write (NFS, Azure Files)
ReadWriteOncePod	RWOP	Single Pod read/write (K8s 1.22+)

Chapter 8: Configuration Management

Kubernetes provides first-class objects for managing application configuration and sensitive data separately from container images. This enables the same image to run in different environments with different configurations.

8.1 ConfigMaps

ConfigMaps store non-sensitive configuration data as key-value pairs. They can be consumed by Pods as environment variables, command-line arguments, or config files.

```
# Create ConfigMap from literal values
kubectl create configmap app-config \
  --from-literal=DB_HOST=postgres \
  --from-literal=APP_ENV=production

# Create from file
kubectl create configmap nginx-conf --from-file=nginx.conf
```

```
# Using ConfigMap as environment variables
spec:
  containers:
  - name: app
    envFrom:
    - configMapRef:
        name: app-config
  # Or individual keys:
  env:
  - name: DB_HOST
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: DB_HOST
```

8.2 Secrets

Secrets are similar to ConfigMaps but designed for sensitive data (passwords, tokens, certificates). By default, Secrets are stored base64-encoded in etcd (NOT encrypted — use etcd encryption or external secret managers for true security).

Secret Type	Use Case
Opaque (generic)	Arbitrary key-value pairs
kubernetes.io/service-account-token	ServiceAccount tokens
kubernetes.io/dockerconfigjson	Docker registry credentials
kubernetes.io/tls	TLS certificate and key
kubernetes.io/basic-auth	Username and password
kubernetes.io/ssh-auth	SSH private key

```
# Create a generic secret
kubectl create secret generic db-creds \
  --from-literal=username=admin \
  --from-literal=password='S3cr3tP@ss!'
```

□ **NOTE:** For production, use external secret management: HashiCorp Vault, AWS Secrets Manager, or Sealed Secrets (Bitnami). Never commit Secret YAML with actual values to Git.

8.3 Environment Variables

Environment variables are the simplest way to configure containers. They can come from literal values, ConfigMaps, Secrets, or the downward API (Pod metadata):

```
env:
- name: MY_NODE_NAME      # From Pod's own metadata (Downward API)
  valueFrom:
    fieldRef:
      fieldPath: spec.nodeName
- name: MY_CPU_LIMIT      # From resource limits
  valueFrom:
    resourceFieldRef:
      containerName: app
      resource: limits.cpu
```


Chapter 9: Security in Kubernetes

Security in Kubernetes is multi-layered (defense in depth). It spans cluster infrastructure, API server access control, network isolation, and runtime security. The 4Cs of Cloud Native Security form the framework: Cloud, Cluster, Container, Code.

9.1 Authentication

Kubernetes does not have a built-in user database. All API requests carry credentials, and Kubernetes uses pluggable authentication modules:

- X.509 Client Certificates — Most common for cluster components and admin
- Static Token Files — Simple bearer tokens (not recommended for production)
- Bootstrap Tokens — For node bootstrapping
- OIDC (OpenID Connect) — Enterprise SSO integration (Okta, Dex, Azure AD)
- Service Account Tokens — For Pods accessing the API Server

9.2 RBAC (Role-Based Access Control)

RBAC is the primary authorization mechanism in Kubernetes. It uses four key objects:

□ DIAGRAM — RBAC Object Relationships

Draw four boxes: (1) 'ServiceAccount / User / Group' (actor); (2) 'Role' (namespaced, defines permissions: verbs+resources); (3) 'ClusterRole' (cluster-wide, same structure); (4) 'RoleBinding' and 'ClusterRoleBinding'. Draw arrows: Actor → RoleBinding → Role (namespaced). Actor → ClusterRoleBinding → ClusterRole (cluster-wide). Add a table showing: verbs = [get, list, watch, create, update, patch, delete]; resources = [pods, deployments, services, secrets, etc.]

```
# Role - namespace scoped
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: production
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
```

```
# RoleBinding - binds role to subject
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: production
subjects:
- kind: User
  name: jane
  apiGroup: rbac.authorization.k8s.io
```

```
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

9.3 Pod Security

Pod Security Admission (PSA) — the replacement for PodSecurityPolicy — enforces security standards at the namespace level using three built-in profiles:

Profile	Description
Privileged	No restrictions; maximum permissions (for system-level workloads)
Baseline	Prevents known privilege escalations; minimal restrictions
Restricted	Heavily restricted; follows current Pod hardening best practices

```
# Label namespace to enforce restricted profile
kubectl label namespace production pod-security.kubernetes.io/enforce=restricted
```

9.4 Security Contexts

SecurityContext defines privilege and access control settings for a Pod or Container:

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
  containers:
  - name: app
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
        add: ["NET_BIND_SERVICE"]
```

Chapter 10: Scheduling & Node Management

Kubernetes scheduling determines which node each Pod runs on. Understanding scheduling mechanisms lets you optimize workload placement for performance, cost, and reliability.

10.1 Labels and Selectors

Labels are key-value pairs attached to Kubernetes objects. Selectors filter objects by their labels. They are the fundamental mechanism for grouping and selecting resources.

```
# Add labels to a node
kubectl label node node1 disktype=ssd region=us-east-1

# Node selector - schedule on nodes with ssd disks
spec:
  nodeSelector:
    disktype: ssd
```

10.2 Taints and Tolerations

Taints allow nodes to repel a set of Pods. Tolerations allow (but don't require) Pods to schedule on tainted nodes. Together they ensure Pods don't run on inappropriate nodes.

Analogy: A taint is a 'Keep Out' sign on a node. A toleration is a Pod's badge that says 'I'm allowed here.'

```
# Taint a node (only GPU workloads allowed)
kubectl taint node gpu-node1 dedicated=gpu:NoSchedule

# Pod with toleration
spec:
  tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "gpu"
    effect: "NoSchedule"
```

Taint Effect	Behavior
NoSchedule	Pods without toleration are NOT scheduled on the node
PreferNoSchedule	Scheduler tries to avoid, but not guaranteed
NoExecute	Evicts existing Pods without toleration; prevents new scheduling

10.3 Node Affinity

Node Affinity is an advanced version of nodeSelector. It supports more expressive operators (In, NotIn, Exists, DoesNotExist, Gt, Lt) and soft (preferred) vs hard (required) rules:

```
spec:
  affinity:
```

```
nodeAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kubernetes.io/arch
            operator: In
            values: [amd64]
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
            - key: disktype
              operator: In
              values: [ssd]
```

10.4 Pod Affinity and Anti-Affinity

Pod Affinity allows you to constrain which nodes Pods can be scheduled based on labels of other Pods already running on the node. Anti-affinity does the opposite.

- Use Pod Affinity to co-locate Pods that communicate frequently (reduce latency)
- Use Pod Anti-Affinity to spread replicas across nodes (improve availability)

```
# Anti-affinity: spread replicas across nodes
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: nginx
        topologyKey: kubernetes.io/hostname
```

10.5 Resource Quotas and LimitRanges

ResourceQuotas set aggregate limits on the total resources a namespace can consume. LimitRanges set default and min/max constraints for individual containers:

```
# ResourceQuota for a namespace
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
  namespace: team-a
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    count/pods: "20"
    count/services: "10"
```

Chapter 11: Observability — Logging, Metrics & Tracing

Observability is the ability to understand what is happening inside your systems. The three pillars of observability are Logs, Metrics, and Traces. Kubernetes provides built-in foundations for each.

11.1 Logging Architecture

□ DIAGRAM — Kubernetes Logging Architecture

Draw two approaches side-by-side. LEFT - 'Node-Level Logging': Show Pod → Container → stdout/stderr → Node Log File (/var/log/containers/) → DaemonSet (Fluentd/Filebeat) → Centralized Log Storage (Elasticsearch/CloudWatch/Splunk). RIGHT - 'Sidecar Logging': Show Application Container writing to a Shared Volume, and a Sidecar Container reading from same volume and forwarding logs. Label each approach with its use case.

```
# Basic log commands
kubectl logs pod-name           # Current logs
kubectl logs pod-name -f        # Follow/stream logs
kubectl logs pod-name --previous # Logs from previous crash
kubectl logs pod-name -c container-name # Multi-container pod
kubectl logs -l app=nginx --tail=100 # All pods with label
```

11.2 Metrics with Prometheus

Prometheus is the de facto standard for Kubernetes metrics. It scrapes metrics endpoints, stores time-series data, and provides a query language (PromQL). The Kubernetes Prometheus Operator simplifies deployment.

- kube-state-metrics — Exposes Kubernetes object state (Deployment replicas, Pod status)
- node-exporter — Exposes node-level metrics (CPU, memory, disk, network)
- cAdvisor — Built into kubelet; exposes container resource usage
- metrics-server — Lightweight in-cluster metrics for HPA and kubectl top

```
# Useful kubectl metrics commands
kubectl top nodes           # Node resource usage
kubectl top pods --all-namespaces # All pod resource usage
kubectl top pods -l app=nginx  # Pod resource by label
```

11.3 Grafana Dashboards

Grafana visualizes metrics from Prometheus with rich, customizable dashboards. For Kubernetes, the most popular dashboards are:

- Dashboard ID 315 — Kubernetes Cluster Monitoring (via Prometheus)
- Dashboard ID 13646 — Kubernetes views for Pods
- Dashboard ID 8588 — Kubernetes Deployment Statefulset Daemonset metrics

11.4 Distributed Tracing with Jaeger/Zipkin

Distributed tracing tracks a request as it flows through multiple microservices. Each service adds spans to a trace, enabling you to identify bottlenecks and errors in complex distributed systems.

- OpenTelemetry — Vendor-neutral instrumentation standard
- Jaeger — CNCF tracing backend from Uber
- Zipkin — Alternative tracing backend from Twitter

11.5 Health Probes

Kubernetes uses three types of health probes to determine container health:

Probe	Purpose	Failure Action
livenessProbe	Is the container alive? (deadlock check)	Kill and restart container
readinessProbe	Is the container ready to serve traffic?	Remove from Service endpoints
startupProbe	Has the container started successfully?	Kill container if startup fails

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
```

Chapter 12: Helm — Kubernetes Package Manager

Helm is the package manager for Kubernetes, similar to apt for Ubuntu or brew for macOS. It uses 'charts' — packages of pre-configured Kubernetes resources — to deploy applications.

12.1 Helm Concepts

Concept	Description
Chart	A Helm package; a collection of YAML templates and default values
Release	A running instance of a chart in a Kubernetes cluster
Repository	A collection of charts (like Artifact Hub)
Values	Configuration overrides for a chart
Templates	Go template files that generate Kubernetes manifests
Hooks	Actions to perform at specific points in a release lifecycle

12.2 Essential Helm Commands

```
# Add a chart repository
helm repo add stable https://charts.helm.sh/stable
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Search for charts
helm search repo nginx
helm search hub wordpress

# Install a chart
helm install my-nginx bitnami/nginx
helm install my-wordpress bitnami/wordpress --set wordpressUsername=admin
helm install my-app ./my-chart --values custom-values.yaml

# Manage releases
helm list --all-namespaces
helm upgrade my-nginx bitnami/nginx --set replicaCount=3
helm rollback my-nginx 1
helm uninstall my-nginx
```

12.3 Creating a Custom Chart

```
# Generate chart scaffold
helm create my-app
```

Chart directory structure:

```
my-app/
```

```
|— Chart.yaml          # Chart metadata (name, version, description)
|— values.yaml         # Default configuration values
|— templates/          # Kubernetes manifest templates
|   |— deployment.yaml
|   |— service.yaml
|   |— ingress.yaml
|   |— _helpers.tpl    # Template helpers/partials
|   └─ NOTES.txt       # Post-install notes
|— charts/             # Chart dependencies
└─ .helmignore
```

```
# Validate and debug templates
helm lint my-app/
helm template my-app my-app/ --values prod-values.yaml
helm install my-app my-app/ --dry-run --debug
```


Chapter 13: CI/CD and GitOps with Kubernetes

Modern Kubernetes deployments use CI/CD pipelines to automate testing and deployment, and GitOps to manage cluster state declaratively using Git as the single source of truth.

13.1 CI/CD Pipeline Overview

□ DIAGRAM — Kubernetes CI/CD Pipeline

Draw a horizontal pipeline with stages: (1) 'Developer commits code' → (2) 'Git Repository (GitHub/GitLab)' → (3) 'CI System (GitHub Actions / Jenkins / GitLab CI)' containing: Build Image → Run Tests → Push to Registry → (4) 'Container Registry (ECR/GCR/Docker Hub)' → (5) 'CD System' → (6) 'Kubernetes Cluster' (with Dev, Staging, Production namespaces). Add arrows for the complete flow. Show GitOps branch where ArgoCD watches Git repo and syncs to cluster.

13.2 GitHub Actions for Kubernetes

```
# .github/workflows/deploy.yaml
name: Build and Deploy
on:
  push:
    branches: [main]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build Docker image
        run: |
          docker build -t myapp:${{ github.sha }} .
          docker push myapp:${{ github.sha }}
      - name: Deploy to Kubernetes
        run: |
          kubectl set image deployment/myapp \
            app=myapp:${{ github.sha }} \
            --record
```

13.3 GitOps with ArgoCD

ArgoCD is a declarative GitOps continuous delivery tool for Kubernetes. It continuously monitors a Git repository and automatically syncs the cluster state to match the desired state defined in Git.

- Installs as a Kubernetes controller inside the cluster
- Monitors Git repositories for changes
- Automatically (or manually) syncs cluster state to desired state
- Detects and alerts on configuration drift
- Provides rollback via Git revert

```
# Install ArgoCD
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml

# Create an ArgoCD Application
argocd app create my-app \
  --repo https://github.com/org/k8s-configs \
  --path production/my-app \
  --dest-server https://kubernetes.default.svc \
  --dest-namespace production \
  --sync-policy automated
```

13.4 FluxCD — Alternative GitOps

FluxCD is another CNCF GitOps tool that works similarly to ArgoCD but has a different architecture. It uses Kubernetes controllers that pull changes from Git rather than a central server pushing changes.

Feature	ArgoCD	FluxCD
Architecture	Server-based with UI	Controller-based, GitOps-native
UI	Rich built-in dashboard	External Weave GitOps UI
Multi-cluster	Good support	Good support
CNCF Status	Graduated	Graduated
Learning Curve	Medium	Medium

Chapter 14: Advanced Topics

14.1 Custom Resource Definitions (CRDs)

CRDs extend the Kubernetes API with custom object types. They enable you to define your own resource types and have Kubernetes manage them like built-in resources. Operators use CRDs extensively.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.mycompany.io
spec:
  group: mycompany.io
  names:
    kind: Database
    plural: databases
    singular: database
    shortNames: [db]
  scope: Namespaced
  versions:
    - name: v1alpha1
      served: true
      storage: true
```

14.2 Kubernetes Operators

Operators are Kubernetes-native applications that use CRDs and custom controllers to manage complex, stateful applications. They encode operational knowledge into software.

- Database Operators — PostgreSQL Operator (Zalando), MySQL Operator, MongoDB Community Operator
- Observability — Prometheus Operator, Grafana Operator
- Security — cert-manager, Vault Operator
- Build Tools — Operator SDK (Go, Helm, Ansible)

14.3 Service Mesh

A service mesh provides Layer 7 networking features between services without modifying application code. It uses sidecar proxies (typically Envoy) injected into each Pod.

Feature	Description
Mutual TLS (mTLS)	Automatic encryption and authentication between services
Traffic Management	Fine-grained routing, retries, timeouts, circuit breaking
Observability	Automatic distributed tracing and metrics
Authorization	Service-level access policies

Popular Tools

Istio, Linkerd, Consul Connect, Cilium

14.4 Multi-Cluster Management

Large organizations often run multiple Kubernetes clusters for isolation, multi-region deployments, or separation of concerns. Tools for multi-cluster management:

- Cluster API — Declarative cluster lifecycle management using Kubernetes itself
- Rancher — Enterprise multi-cluster management UI and platform
- KubeFed (Federation v2) — Multi-cluster resource federation
- ArgoCD ApplicationSets — Deploy to multiple clusters from one ArgoCD instance
- Submariner — Cross-cluster networking and service discovery

14.5 Vertical Pod Autoscaler (VPA)

VPA automatically adjusts CPU and memory requests/limits for containers based on actual usage. It complements HPA: HPA scales out (more replicas), VPA scales up (bigger containers).

□ **NOTE:** Do not use HPA and VPA together on the same resource metric (CPU/memory). This can cause conflicts. Use HPA for scale-out based on custom metrics, and VPA for right-sizing.

14.6 Cluster Autoscaler

Cluster Autoscaler automatically adjusts the size of a Kubernetes cluster by adding nodes when Pods cannot be scheduled and removing nodes when they are consistently underutilized. It integrates with cloud provider node groups (AWS ASG, GCP MIG, Azure VMSS).

Chapter 15: Production Best Practices

Running Kubernetes in production requires careful attention to reliability, security, and cost. This chapter consolidates the most important production best practices.

15.1 Reliability Best Practices

- Always use Deployments (not bare Pods) for application workloads
- Set meaningful resource requests AND limits for every container
- Configure liveness and readiness probes for all application containers
- Use Pod Disruption Budgets (PDB) to prevent too many pods being down during maintenance
- Set `terminationGracePeriodSeconds` appropriately (default 30s is often too low)
- Use multiple replicas across multiple availability zones
- Implement retry logic and circuit breakers in your applications

15.2 Security Best Practices

- Enable RBAC and grant least-privilege access
- Scan container images for vulnerabilities (Trivy, Snyk, Gype)
- Never run containers as root; use non-root UIDs
- Set `readOnlyRootFilesystem: true` where possible
- Use Network Policies to restrict Pod-to-Pod communication
- Rotate and externalize secrets (Vault, AWS SM, Sealed Secrets)
- Enable audit logging on the API Server
- Use admission controllers (OPA/Gatekeeper, Kyverno) for policy enforcement

15.3 Cost Optimization

- Use the Cluster Autoscaler to right-size node pools
- Use Spot/Preemptible instances for fault-tolerant workloads (with proper tolerations)
- Set VPA recommendations to right-size container resource requests
- Use namespace ResourceQuotas to prevent resource hoarding
- Implement Goldilocks (VPA in recommendation mode) for right-sizing
- Delete unused PersistentVolumes and LoadBalancer Services

15.4 Upgrade Strategy

Kubernetes releases a new minor version every 4 months. Each minor version is supported for 14 months. Best practices for upgrades:

- Never skip minor versions — upgrade one minor version at a time (1.27 → 1.28 → 1.29)
- Upgrade the Control Plane before worker nodes
- Review API deprecations for each version (use `kubectl convert`, `pluto`)
- Test upgrades in non-production clusters first
- Use `PodDisruptionBudgets` to ensure service continuity during node drain

- Managed services (EKS, GKE, AKS) handle the control plane upgrade for you

15.5 Disaster Recovery

- Regularly backup etcd using etcdctl snapshot save
- Use Velero for backup and restore of Kubernetes resources and PersistentVolumes
- Test your restore procedure regularly — backups are only as good as a successful restore
- Document your RTO (Recovery Time Objective) and RPO (Recovery Point Objective)
- Consider multi-region cluster federation for critical workloads

15.6 Kubectl Productivity Tips

```
# Aliases (add to ~/.bashrc or ~/.zshrc)
alias k=kubectl
alias kgp='kubectl get pods'
alias kgs='kubectl get services'
alias kgd='kubectl get deployments'
alias kns='kubectl config set-context --current --namespace'

# Useful one-liners
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get all -n production
kubectl exec -it pod-name -- /bin/bash
kubectl port-forward svc/my-service 8080:80
kubectl diff -f manifest.yaml
kubectl api-resources --verbs=list --namespaced -o name
```



End of Kubernetes Textbook

[tech__talks](#)

Chapters 1–15 | Architecture • Workloads • Networking • Storage • Security • Helm • CI/CD • Production